



**Team
AXIOMATA**

PLANNING ENGINE

Developed By
Team AXIOMATA

Developed for
Waypoint Group (Pvt) Ltd.

Access to RELAY

1. Overview

Relay's planning engine is a pure Python package (`packages/engine/relay_engine`) built on Google OR-Tools CP-SAT. It takes one service day's confirmed orders, the available vehicles, and the published planning standard, and produces an allocation that satisfies every hard rule, an ordered stop sequence and clock times for every trip, and an explanation for each deferred order. The same `plan()` function powers the live app and the Datathon Task 2B peak-day submission.

2. Inputs & Outputs

2.1 Inputs (`build_problem`)

Input	Source
Orders (confirmed / planned / deferred)	Database - brand, temperature, weight, volume, outlet window, district, usual vehicle
Vehicles	vehicles.csv - type, temp, weight/volume capacity, km per litre, weekly fuel quota, depot
Planning standard	district_travel.csv (travel minutes/km), service_allowance.csv (handling minutes)
Deferral policy	Dispatcher selection: Max Throughput / Balanced / Fairness First
Pinned trips	Trips the dispatcher has locked - engine cannot reassign their orders

2.2 Outputs (Solution)

Output	Description
assignments	order $\rightarrow$ (vehicle, trip 1 or 2)
trips	Per trip: stop sequence, departure, arrivals, start/finish, return, minutes/km/litres/m ³ /kg
deferred	One Deferral (kind, code, text, cost) for every unserved order
kpis	Orders, served, deferred, m ³ chilled, repeat skips, trips, vehicles, km, litres, late stops- per depot and total

3. Hard Constraints

Stage 1 (CP-SAT) makes every rule a hard constraint. `validate_allocation()` checks the result independently, acting as a port of the organisers' `check_allocation.py`.

Rule	Enforcement
Whole orders, each served at most once	AddAtMostOne over each order's assignment variables
No workshop vehicles	Unavailable vehicles excluded from the model entirely
Vehicles serve their home depot only	One model per depot; cross-depot assignments impossible
One brand + one district per trip	Combo variables $y[v,t,c]$; exactly one active combo per used trip slot
Chilled goods on reefers only	x variables created only for compatible vehicle-order pairs
Van-only outlets by van only	No assignment variable for truck-outlet pairs
Volume & weight per trip	Linear constraints on summed demand with integer scaling
At most two trips per vehicle	Trip slot $t \in \{1, 2\}$ only
Fresh window ≤ 270 min (03:30–08:00)	Per-vehicle sum of Fresh trip minutes $\leq$ budget_fresh

Style/Tech window ≤ 480 min	Per-vehicle sum of other trip minutes $\leq$ budget_day
Weekly fuel quota	Per-vehicle km (return included) $\leq$ fuel_left_l $\times$ km_per_l

4. Two-Stage Solve

4.1 Stage 1 - CP-SAT Allocation

One CP-SAT model is built per depot (depots never share vehicles, so they are independent). A greedy warm start assigns orders to vehicles in decreasing volume order; this is fed as a hint to speed convergence. The solver runs with a time limit; the gap from optimal is reported in the KPIs. Depots are solved in parallel threads.

4.2 Stage 2 - Stop Sequence & Clock Times

Given the allocation from Stage 1, timetable.py orders the stops within each trip using the planning-standard travel times. It computes the departure, each stop's planned arrival start and finish, and the return time. Stops with a narrow delivery window are sequenced to minimise lateness. Late flags are set where the window cannot be met.

5. Deferral Explanations

Every deferred order is explained as either unavoidable (no vehicle could ever serve it under the rules) or a choice (it could be served, but doing so would displace other orders or break constraints). The explain module runs counterfactual re-solves: it forces each deferred order on, re-plans, and reports the cost (orders displaced, extra km, chilled m³ lost). This gives the dispatcher actionable information rather than a bare list of misses.

6. Deferral Lever — Three Policies

Policy	Objective
Max Throughput	Maximise total orders served and m ³ delivered
Balanced	Balance served orders across the two depots
Fairness First	Minimise repeat skips - prioritise outlets that have waited longest

The dispatcher can preview all three policies on the current day's draft before committing. Switching policy triggers a new solve; the deferral lever panel shows the KPI comparison side by side.

7. Repair Pass

After the explanation pass, the engine checks whether any deferred order would fit on an existing trip without displacing any other order. If so, it re-solves once with those orders forced on, redoes Stage 2 and re-explains. This catches cases where the initial solve left unused capacity.

8. ML: Stop-Time Prediction

relay_ml (packages/ml) wraps a LightGBM model trained on historical delivery data. For each planned stop it predicts:

- Pred_service_min → expected service time in minutes
- late_prob → probability of arriving after the delivery window

Features include district, brand, dock type, parking, day-of-week, hour, traffic speed index (from traffic_speed.csv), vehicle type and load. A heuristic fallback (based on service_allowance.csv values) is used when no trained model is available. Predictions are attached to each stop after the plan is persisted and are visible on the dispatcher's canvas as risk indicators.

9. Live ETA during Execution

While a run is in progress, domain/eta.py recomputes ETAs for remaining stops using the actual departure time and any completed stops' arrival records. The dispatcher and store manager see 'Estimated' times that update in real time as the driver reports in. When a van is dark (no signal), ETAs are projected from the last known position using planning-standard travel times.

10. Validation

validate_allocation() is a faithful port of the organisers' check_allocation.py with an additional fuel-quota check. It runs on every test plan in the unit-test suite and in CI ('make check-plan'). A capacity rounding error discovered by this checker during development was fixed and a regression test added.